RDF 1.2 Schema

W3C Working Draft 26 December 2025



▼ **More details about this document**

This version:

https://www.w3.org/TR/2025/WD-rdf12-schema-20251226/

Latest published version:

https://www.w3.org/TR/rdf12-schema/

Latest editor's draft:

https://w3c.github.io/rdf-schema/spec/

History:

https://www.w3.org/standards/history/rdf12-schema/

Commit history

Latest Recommendation:

https://www.w3.org/TR/rdf11-schema/

Editor:

Dominik Tomaszuk

Former editors:

Guus Schreiber

Dan Brickley

R.V. Guha

Brian McBride

Feedback:

GitHub w3c/rdf-schema (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[rdf12-schema] … message topic …` (archives)

Copyright © 2004-2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

RDF Schema provides a data-modelling vocabulary for RDF data. RDF Schema is an extension of the basic RDF vocabulary.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is an edited version of the 2014 RDF Schema Recommendation. The purpose of this revision is to make this document available as part of the RDF 1.2 document set. Changes are limited to errata, revised references, terminology updates, and adaptations to the introduction. The title of the document was changed from *RDF Schema 1.1* to *RDF 1.2 Schema*. The technical content of the document is unchanged. Details of the changes are listed in the Changes section. Since the edits to this document do not constitute a technical change the Director decided no new implementation report was required.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▼ List of documents

RDF 1.2 Documents:

1. *What's New in RDF 1.2*
2. *RDF 1.2 Concepts and Abstract Data Model*
3. *RDF 1.2 N-Quads*
4. *RDF 1.2 N-Triples*
5. *RDF 1.2 Primer*
6. RDF 1.2 Schema (this document)
7. *RDF 1.2 Semantics*
8. *RDF 1.2 TriG*
9. *RDF 1.2 Turtle*
10. *RDF 1.2 XML Syntax*
11. *RDF 1.2 Interoperability*

SPARQL 1.2 Documents:

1. *What's New in SPARQL 1.2*
2. *SPARQL 1.2 Concepts*
3. *SPARQL 1.2 Query Language*
4. *SPARQL 1.2 Update*
5. *SPARQL 1.2 Service Description*
6. *SPARQL 1.2 Federated Query*
7. *SPARQL 1.2 Query Results JSON Format*
8. *SPARQL 1.2 Query Results CSV and TSV Formats*
9. *SPARQL 1.2 Query Results XML Format*
10. *SPARQL 1.2 Entailment Regimes*

11. *SPARQL 1.2 Protocol*
12. *SPARQL 1.2 Graph Store Protocol*

Table of Contents

Abstract

Status of This Document

Set of Documents

1. Introduction

2. Data-modelling vocabulary

2.1 Classes

2.1.1 `rdfs:Resource`

2.1.2 `rdfs:Class`

2.1.3 `rdfs:Literal`

2.1.4 `rdfs:Datatype`

2.1.5 `rdf:Property`

2.2 Semantic Properties

2.2.1 `rdfs:range`

2.2.2 `rdfs:domain`

2.2.3 `rdf:type`

2.2.4 `rdfs:subClassOf`

2.2.5 `rdfs:subPropertyOf`

2.3 Documentation Properties

2.3.1 `rdfs:label`

2.3.2 `rdfs:comment`

2.3.3 `rdfs:seeAlso`

2.3.4 `rdfs:isDefinedBy`

2.4 Using the Domain and Range vocabulary

3. Datatypes

3.1 `rdf:langString`

3.2 `rdf:dirLangString`

3.3 rdf:HTML
3.4 rdf:XMLLiteral
3.5 rdf:JSON
3.6 XML Schema datatypes

4. RDF Collections
4.1 rdf:List
4.2 rdf:first
4.3 rdf:rest
4.4 rdf:nil

5. Triple-term based reification
5.1 rdfs:Proposition
5.2 rdf:reifies

6. Utility Properties
6.1 rdf:value

7. Legacy vocabularies
7.1 Containers
7.1.1 rdfs:Container
7.1.2 rdf:Bag
7.1.3 rdf:Seq
7.1.4 rdf:Alt
7.1.5 rdfs:ContainerMembershipProperty
7.1.6 rdfs:member
7.2 "Old-style" Reification
7.2.1 rdf:Statement
7.2.2 rdf:subject
7.2.3 rdf:predicate
7.2.4 rdf:object

8. RDF Schema summary
8.1 RDF classes
8.2 RDF properties

A. Vocabulary Registry

A.1 `rdf:` Namespace
A.2 `rdfs:` Namespace

B. Privacy Considerations

C. Security Considerations

D. Acknowledgments
D.1 Acknowledgments for 2004 Recommendation
D.2 Acknowledgments for RDF 1.1
D.3 Acknowledgments for RDF 1.2

E. Change since 2004 Recommendation

F. Changes between RDF 1.1 and RDF 1.2

G. Index
G.1 Terms defined by this specification
G.2 Terms defined by reference

H. References
H.1 Informative references

§ 1. Introduction

RDF Schema provides a data-modelling vocabulary for RDF data. It is complemented by several companion documents which describe the basic concepts and abstract syntax of RDF [*RDF12-CONCEPTS*], the formal semantics of RDF [*RDF12-SEMANTICS*], and various concrete syntaxes for RDF, such as Turtle [*RDF12-TURTLE*], TriG, [*RDF12-TRIG*], and JSON-LD [*JSON-LD11*]. The RDF Primer [*RDF12-PRIMER*] provides an informal introduction and examples of the use of the concepts specified in this document.

This document serves two complementary purposes:

- It defines RDF Schema as a vocabulary for specifying lightweight ontologies, underlying the semantic extension described in RDF Semantics (§9 of [*RDF12-SEMANTICS*]).

- It enumerates all terms defined within the `rdf:` and `rdfs:` namespaces, including both terms relevant to ontology construction and general-purpose terms used broadly within RDF applications.

This document is intended to provide a clear specification of RDF Schema to those who find the formal semantics specification [*RDF12-SEMANTICS*] daunting. Thus, this document duplicates material also specified in the RDF Semantics specification. Where there is disagreement between this document and the RDF Semantics specification, the RDF Semantics specification should be taken to be correct.

RDF Schema is a semantic extension of RDF. It provides mechanisms for describing groups of related resources and the relationships between these resources. RDF Schema is written in RDF using the terms described in this document. These resources are used to determine characteristics of other resources, such as the domains and ranges of properties.

The RDF Schema class and property system is similar to the type systems of object-oriented programming languages such as Java. RDF Schema differs from many such systems in that instead of defining a class in terms of the properties its instances may have, RDF Schema describes properties in terms of the classes of resource to which they apply. This is the role of the domain and range mechanisms described in this specification. For example, we could define the `eg:author` property to have a domain of `eg:Document` and a range of `eg:Person`, whereas a classical object oriented system might typically define a class `eg:Book` with an attribute called `eg:author` of type `eg:Person`. Using the RDF approach, it is easy for others to subsequently define additional properties with a domain of `eg:Document` or a range of `eg:Person`. This can be done without the need to re-define the original description of these classes. One benefit of the RDF property-centric approach is that it allows anyone to extend the description of existing resources, one of the architectural principles of the Web [*RDF-NOT*].

However, RDF Schema also includes numerous terms in the *rdf:* and *rdfs:* namespaces that support broader RDF functionality and general data representation needs. These terms are not exclusively tied to ontology specification but serve essential roles in RDF's broader architecture.

This specification does not attempt to enumerate all the possible forms of representing the meaning of RDF classes and properties. Instead, the RDF Schema strategy is to acknowledge that there are many techniques through which the meaning of classes and properties can be described. Richer vocabulary or 'ontology' languages such as OWL [*OWL2-OVERVIEW*],

inference rule languages and other formalisms (for example temporal logics) will each contribute to our ability to capture meaningful generalizations about data in the Web.

The language defined in this specification consists of a collection of RDF resources that can be used to describe other RDF resources in application-specific RDF vocabularies. The core vocabulary is defined in a namespace informally called `rdfs` here.

The IRIs in an RDF vocabulary often begin with a common substring known as a namespace IRI. Some namespace IRIs are associated by convention with a short name known as a namespace prefix.

Some namespace prefixes and IRIs used in this specification

Namespace prefix	**Namespace IRI**
rdf	`http://www.w3.org/1999/02/22-rdf-syntax-ns#`
rdfs	`http://www.w3.org/2000/01/rdf-schema#`
xsd	`http://www.w3.org/2001/XMLSchema#`

For convenience and readability, this specification uses an abbreviated form to represent IRIs. A name of the form `prefix:suffix` should be interpreted as a IRI consisting of the IRI associated with the `prefix` concatenated with the `suffix`.

§ 2. Data-modelling vocabulary

§ 2.1 Classes

Resources may be divided into groups called classes. The members of a class are known as *instances* of the class. Classes are themselves resources. They are often identified by IRIs and may be described using RDF properties. The `rdf:type` property may be used to state that a resource is an instance of a class.

RDF distinguishes between a class and the set of its instances. Associated with each class is a set, called the class extension of the class, which is the set of the instances of the class. Two

classes may have the same set of instances but be different classes. For example, the tax office may define the class of people living at the same address as the editor of this document. The Post Office may define the class of people whose address has the same zip code as the address of the author. It is possible for these classes to have exactly the same instances, yet to have different properties. Only one of the classes has the property that it was defined by the tax office, and only the other has the property that it was defined by the Post Office.

A class may be a member of its own class extension and may be an instance of itself.

The group of resources that are RDF Schema classes is itself a class called `rdfs:Class`.

If a class *C* is a *subclass* of a class *C'*, then all instances of *C* will also be instances of *C'*. The `rdfs:subClassOf` property may be used to state that one class is a subclass of another. The term super-class is used as the inverse of subclass. If a class *C'* is a super-class of a class *C*, then all instances of *C* are also instances of *C'*.

The RDF Concepts and Abstract Syntax [*RDF12-CONCEPTS*] specification defines the RDF concept of an RDF datatype. All datatypes are classes. The instances of a class that is a datatype are the members of the value space of the datatype.

§ 2.1.1 `rdfs:Resource`

All things described by RDF are called *resources*, and are instances of the class `rdfs:Resource`. This is the class of everything. All other classes are subclasses of this class. `rdfs:Resource` is an instance of `rdfs:Class`.

§ 2.1.2 `rdfs:Class`

This is the class of resources that are RDF classes. `rdfs:Class` is an instance of `rdfs:Class.`

§ 2.1.3 `rdfs:Literal`

The class `rdfs:Literal` is the class of literal values such as strings and integers. Property values such as textual strings are examples of RDF literals.

`rdfs:Literal` is an instance of `rdfs:Class`. `rdfs:Literal` is a subclass of `rdfs:Resource`.

§ 2.1.4 `rdfs:Datatype`

`rdfs:Datatype` is the class of datatypes. All instances of `rdfs:Datatype` correspond to the RDF model of a datatype described in the RDF Concepts specification [*RDF12-CONCEPTS*]. `rdfs:Datatype` is both an instance of and a subclass of `rdfs:Class`. Each instance of `rdfs:Datatype` is a subclass of `rdfs:Literal`.

§ 2.1.5 `rdf:Property`

`rdf:Property` is the class of RDF properties. `rdf:Property` is an instance of `rdfs:Class`.

§ 2.2 Semantic Properties

The RDF Concepts and Abstract Syntax specification [*RDF12-CONCEPTS*] describes the concept of an RDF property as a relation between subject resources and object resources.

This specification defines the concept of subproperty. The `rdfs:subPropertyOf` property may be used to state that one property is a subproperty of another. If a property *P* is a subproperty of property *P'*, then all pairs of resources which are related by *P* are also related by *P'*. The term super-property is often used as the inverse of subproperty. If a property *P'* is a super-property of a property *P*, then all pairs of resources which are related by *P* are also

related by *P*'. This specification does not define a top property that is the super-property of all properties.

> NOTE
>
> The basic facilities provided by `rdfs:domain` and `rdfs:range` do not provide any direct way to indicate property restrictions that are local to a class. Although it is possible to combine use `rdfs:domain` and `rdfs:range` with sub-property hierarchies, direct support for such declarations are provided by richer Web Ontology languages such as OWL [*OWL2-OVERVIEW*].

2.2.1 `rdfs:range`

`rdfs:range` is an instance of `rdf:Property` that is used to state that the values of a property are instances of one or more classes.

The triple

```
P rdfs:range C
```

states that *P* is an instance of the class `rdf:Property`, that *C* is an instance of the class `rdfs:Class` and that the resources denoted by the objects of triples whose predicate is *P* are instances of the class *C*.

Where *P* has more than one `rdfs:range` property, then the resources denoted by the objects of triples with predicate *P* are instances of all the classes stated by the `rdfs:range` properties.

The `rdfs:range` property can be applied to itself. The `rdfs:range` of `rdfs:range` is the class `rdfs:Class`. This states that any resource that is the value of an `rdfs:range` property is an instance of `rdfs:Class`.

The `rdfs:range` property is applied to properties. This can be represented in RDF using the `rdfs:domain` property. The `rdfs:domain` of `rdfs:range` is the class `rdf:Property`. This states that any resource with an `rdfs:range` property is an instance of `rdf:Property`.

§ 2.2.2 `rdfs:domain`

`rdfs:domain` is an instance of `rdf:Property` that is used to state that any resource that has a given property is an instance of one or more classes.

A triple of the form:

```
P rdfs:domain C
```

states that *P* is an instance of the class `rdf:Property`, that *C* is a instance of the class `rdfs:Class` and that the resources denoted by the subjects of triples whose predicate is *P* are instances of the class C.

Where a property *P* has more than one `rdfs:domain` property, then the resources denoted by subjects of triples with predicate *P* are instances of all the classes stated by the `rdfs:domain` properties.

The `rdfs:domain` property may be applied to itself. The `rdfs:domain` of `rdfs:domain` is the class `rdf:Property`. This states that any resource with an `rdfs:domain` property is an instance of `rdf:Property`.

The `rdfs:range` of `rdfs:domain` is the class `rdfs:Class`. This states that any resource that is the value of an `rdfs:domain` property is an instance of `rdfs:Class`.

§ 2.2.3 `rdf:type`

`rdf:type` is an instance of `rdf:Property` that is used to state that a resource is an instance of a class.

A triple of the form:

```
R rdf:type C
```

states that *C* is an instance of rdfs:Class and *R* is an instance of *C*.

The rdfs:domain of rdf:type is rdfs:Resource. The rdfs:range of rdf:type is rdfs:Class.

§ 2.2.4 rdfs:subClassOf

The property rdfs:subClassOf is an instance of rdf:Property that is used to state that all the instances of one class are instances of another.

A triple of the form:

```
C1 rdfs:subClassOf C2
```

states that *C1* is an instance of rdfs:Class, *C2* is an instance of rdfs:Class and *C1* is a subclass of *C2*. The rdfs:subClassOf property is transitive.

The rdfs:domain of rdfs:subClassOf is rdfs:Class. The rdfs:range of rdfs:subClassOf is rdfs:Class.

§ 2.2.5 rdfs:subPropertyOf

The property rdfs:subPropertyOf is an instance of rdf:Property that is used to state that all resources related by one property are also related by another.

A triple of the form:

```
P1 rdfs:subPropertyOf P2
```

states that *P1* is an instance of rdf:Property, *P2* is an instance of rdf:Property and *P1* is a subproperty of *P2*. The rdfs:subPropertyOf property is transitive.

The rdfs:domain of rdfs:subPropertyOf is rdf:Property. The rdfs:range of rdfs:subPropertyOf is rdf:Property.

§ 2.3 Documentation Properties

§ 2.3.1 `rdfs:label`

`rdfs:label` is an instance of `rdf:Property` that may be used to provide a human-readable version of a resource's name.

A triple of the form:

```
R rdfs:label L
```

states that *L* is a human readable label for *R*.

The `rdfs:domain` of `rdfs:label` is `rdfs:Resource`. The `rdfs:range` of `rdfs:label` is `rdfs:Literal`.

Multilingual labels are supported using the language tagging facility of RDF literals.

§ 2.3.2 `rdfs:comment`

`rdfs:comment` is an instance of `rdf:Property` that may be used to provide a human-readable description of a resource.

A triple of the form:

```
R rdfs:comment L
```

states that *L* is a human readable description of *R*.

The `rdfs:domain` of `rdfs:comment` is `rdfs:Resource`. The `rdfs:range` of `rdfs:comment` is `rdfs:Literal`.

A textual comment helps clarify the meaning of RDF classes and properties. Such in-line documentation complements the use of both formal techniques (Ontology and rule languages) and informal (prose documentation, examples, test cases). A variety of documentation forms can be combined to indicate the intended meaning of the classes and properties described in an RDF vocabulary. Since RDF vocabularies are expressed as RDF graphs, vocabularies defined in other namespaces may be used to provide richer documentation.

Multilingual documentation is supported through use of the language tagging facility of RDF literals.

§ 2.3.3 `rdfs:seeAlso`

`rdfs:seeAlso` is an instance of `rdf:Property` that is used to indicate a resource that might provide additional information about the subject resource.

A triple of the form:

```
S rdfs:seeAlso O
```

states that the resource *O* may provide additional information about *S*. It may be possible to retrieve representations of *O* from the Web, but this is not required. When such representations may be retrieved, no constraints are placed on the format of those representations.

The `rdfs:domain` of `rdfs:seeAlso` is `rdfs:Resource`. The `rdfs:range` of `rdfs:seeAlso` is `rdfs:Resource`.

§ 2.3.4 `rdfs:isDefinedBy`

`rdfs:isDefinedBy` is an instance of `rdf:Property` that is used to indicate a resource defining the subject resource. This property may be used to indicate an RDF vocabulary in which a resource is described.

A triple of the form:

```
S rdfs:isDefinedBy O
```

states that the resource *O* defines *S*. It may be possible to retrieve representations of *O* from the Web, but this is not required. When such representations may be retrieved, no constraints are placed on the format of those representations. `rdfs:isDefinedBy` is a subproperty of `rdfs:seeAlso`.

The `rdfs:domain` of `rdfs:isDefinedBy` is `rdfs:Resource`. The `rdfs:range` of `rdfs:isDefinedBy` is `rdfs:Resource`.

§ 2.4 Using the Domain and Range vocabulary

This section is non-normative.

This specification introduces an RDF vocabulary for describing the meaningful use of properties and classes in RDF data. For example, an RDF vocabulary might describe limitations on the types of values that are appropriate for some property, or on the classes to which it makes sense to ascribe such properties.

RDF Schema provides a mechanism for describing this information, but does not say whether or how an application should use it. For example, while an RDF vocabulary can assert that an `author` property is used to indicate resources that are instances of the class `Person`, it does not say whether or how an application should act in processing that range information. Different applications will use this information in different ways. For example, data checking tools might use this to help discover errors in some data set, an interactive editor might suggest appropriate values, and a reasoning application might use it to infer additional information from instance data.

RDF vocabularies can describe relationships between vocabulary items from multiple independently developed vocabularies. Since IRIs are used to identify classes and properties on the Web, it is possible to create new properties that have a `domain` or `range` whose value is a class defined in another namespace.

§ 3. Datatypes

An RDF datatype is a class of literal values with an associated value space, lexical space, and lexical-to-value mapping, identified by a datatype IRI [*RDF12-CONCEPTS*].

RDF 1.2 recognizes both RDF-native datatypes (e.g., `rdf:langString`, `rdf:dirLangString`, `rdf:HTML`, `rdf:XMLLiteral`, `rdf:JSON`) and the RDF-compatible subset of XML Schema 1.1 datatypes, which are enumerated below.

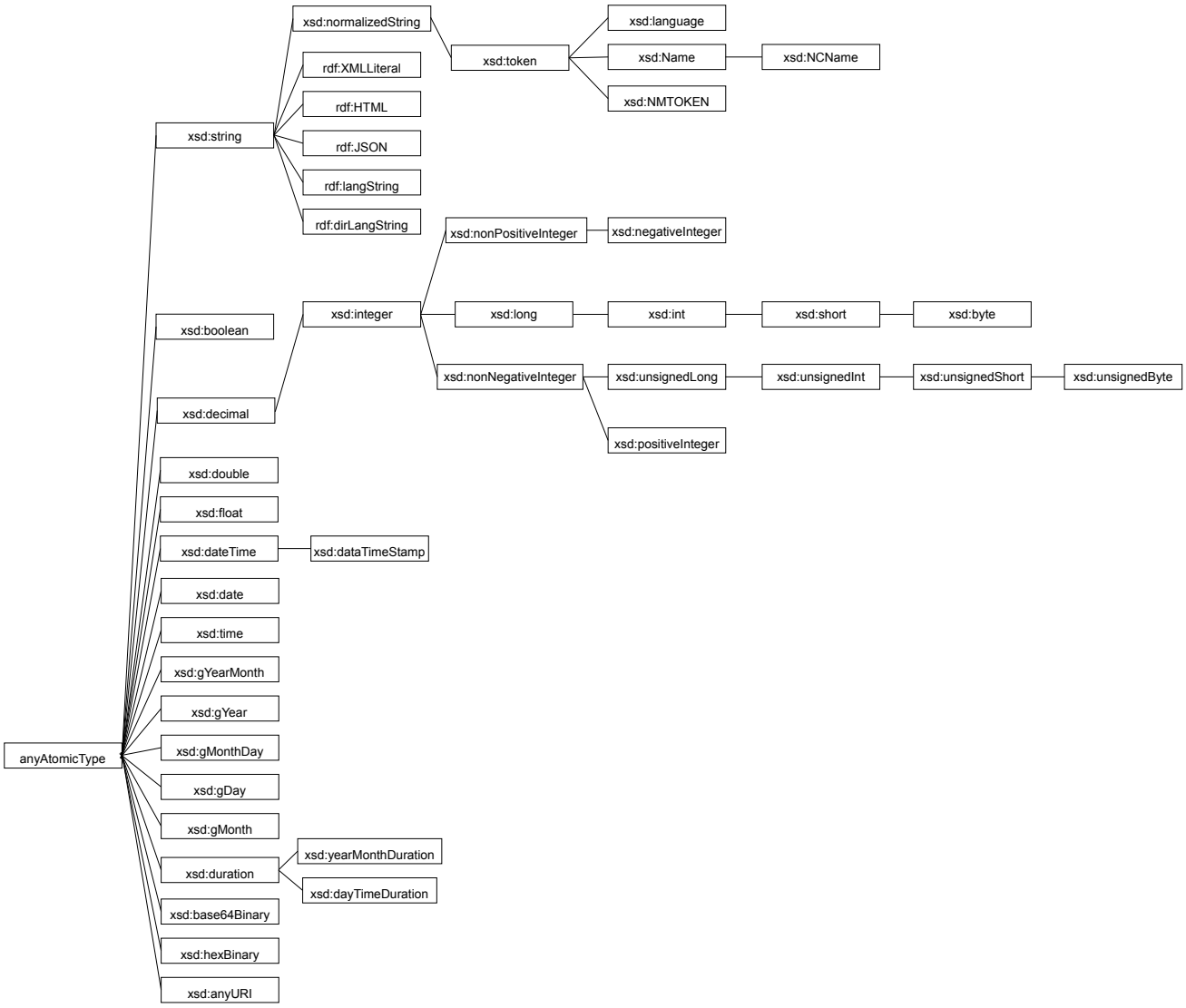


Figure 1 Built-in datatypes hierarchy illustrating the relationships among primitive and derived data types defined in XML Schema and RDF.

§ 3.1 rdf:langString

The class rdf:langString is the class of language-tagged string values. rdf:langString is an instance of rdfs:Datatype and a subclass of rdfs:Literal.

§ 3.2 rdf:dirLangString

The class rdf:dirLangString is the class of directional language-tagged string values. rdf:dirLangString is an instance of rdfs:Datatype and a subclass of rdfs:Literal.

§ 3.3 rdf:HTML

This section is non-normative.

The class rdf:HTML is the class of HTML literal values. rdf:HTML is an instance of rdfs:Datatype and a subclass of rdfs:Literal.

§ 3.4 rdf:XMLLiteral

This section is non-normative.

The class rdf:XMLLiteral is the class of XML literal values. rdf:XMLLiteral is an instance of rdfs:Datatype and a subclass of rdfs:Literal.

§ 3.5 rdf:JSON

This section is non-normative.

The class `rdf:JSON` is the class of JSON literal values. `rdf:JSON` is an instance of `rdfs:Datatype` and a subclass of `rdfs:Literal`.

§ 3.6 XML Schema datatypes

All built-in datatypes defined in the W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes [*XMLSCHEMA11-2*] are identified using IRIs of the form `http://www.w3.org/2001/XMLSchema#xxx`, where `xxx` is the local name of the datatype. Among the full set of XSD datatypes, the subset listed below comprises those that are compatible with the RDF data model and are known as RDF-compatible XSD datatypes [*RDF12-CONCEPTS*].

- Core types
  - `xsd:string`
  - `xsd:boolean`
  - `xsd:decimal`
  - `xsd:integer`
- IEEE floating-point numbers
  - `xsd:double`
  - `xsd:float`
- Time and date
  - `xsd:date`
  - `xsd:time`
  - `xsd:dateTime`
  - `xsd:dateTimeStamp`
- Recurring and partial dates
  - `xsd:gYear`
  - `xsd:gMonth`

    - `xsd:gDay`
    - `xsd:gYearMonth`
    - `xsd:gMonthDay`
- Duration
    - `xsd:duration`
    - `xsd:yearMonthDuration`
    - `xsd:dayTimeDuration`
- Limited-range integer numbers
    - `xsd:byte`
    - `xsd:short`
    - `xsd:int`
    - `xsd:long`
    - `xsd:unsignedByte`
    - `xsd:unsignedShort`
    - `xsd:unsignedInt`
    - `xsd:unsignedLong`
    - `xsd:positiveInteger`
    - `xsd:nonNegativeInteger`
    - `xsd:negativeInteger`
    - `xsd:nonPositiveInteger`
- Encoded binary data
    - `xsd:hexBinary`
    - `xsd:base64Binary`
- Miscellaneous XSD types
    - `xsd:anyURI`
    - `xsd:language`
    - `xsd:normalizedString`
    - `xsd:token`

- `xsd:NMTOKEN`
- `xsd:Name`
- `xsd:NCName`

§ 4. RDF Collections

This section is non-normative.

RDF containers are open in the sense that the core RDF specifications define no mechanism to state that there are no more members. The RDF Collection vocabulary of classes and properties can describe a closed collection, i.e. one that can have no more members.

A collection is represented as a list of items, a representation that will be familiar to those with experience of Lisp and similar programming languages. There is a shorthand notation in the Turtle syntax specification for representing collections.

> NOTE
>
> RDFS does not require that there be only one first element of a list-like structure, or even that a list-like structure have a first element.

RDF provides a vocabulary for describing collections, i.e.'list structures', in terms of head-tail links. Collections differ from containers in allowing branching structure and in having an explicit terminator, allowing applications to determine the exact set of items in the collection.

As with containers, no special semantic conditions are imposed on this vocabulary other than the type of `rdf:nil` being `rdf:List`. It is intended for use typically in a context where a container is described using blank nodes to connect a 'well-formed' sequence of items, each described by two triples of the form

```
_:c1 rdf:first aaa .
_:c1 rdf:rest _:c2 .
```

where the final item is indicated by the use of `rdf:nil` as the value of the property `rdf:rest`. In a familiar convention, `rdf:nil` can be thought of as the empty collection. Any such graph

amounts to an assertion that the collection exists, and since the members of the collection can be determined by inspection, this is often sufficient to enable applications to determine what is meant. The semantics does not require any collections to exist other than those mentioned explicitly in a graph (and the empty collection). For example, the existence of a collection containing two items does not automatically guarantee that the similar collection with the items permuted also exists:

```
_:c1 rdf:first ex:aaa .
_:c1 rdf:rest _:c2 .
_:c2 rdf:first ex:bbb .
_:c2 rdf:rest rdf:nil .
```

does not entail

```
_:c3 rdf:first ex:bbb .
_:c3 rdf:rest _:c4 .
_:c4 rdf:first ex:aaa .
_:c4 rdf:rest rdf:nil .
```

Also, RDF imposes no 'well-formedness' conditions on the use of this vocabulary, so that it is possible to write RDF graphs which assert the existence of highly peculiar objects such as lists with forked or non-list tails, or multiple heads:

```
_:666 rdf:first ex:aaa .
_:666 rdf:first ex:bbb .
_:666 rdf:rest ex:ccc .
_:666 rdf:rest rdf:nil .
```

It is also possible to write a set of triples which under-specify a collection by failing to specify its `rdf:rest` property value.

Semantic extensions may place extra syntactic well-formedness restrictions on the use of this vocabulary in order to rule out such graphs. They may exclude interpretations of the collection vocabulary which violate the convention that the subject of a 'linked' collection of two-triple items of the form described above, ending with an item ending with `rdf:nil`, denotes a totally ordered sequence whose members are the referents of the `rdf:first` values of the items, in the order got by tracing the `rdf:rest` properties from the subject to `rdf:nil`. This permits sequences which contain other sequences.

The RDFS semantic conditions require that any subject of the `rdf:first` property, and any subject or object of the `rdf:rest` property, be of `rdf:type rdf:List`.

§ 4.1 `rdf:List`

`rdf:List` is an instance of `rdfs:Class` that can be used to build descriptions of lists and other list-like structures.

§ 4.2 `rdf:first`

`rdf:first` is an instance of `rdf:Property` that can be used to build descriptions of lists and other list-like structures.

A triple of the form:

```
L rdf:first O
```

states that there is a first-element relationship between *L* and *O*.

The `rdfs:domain` of `rdf:first` is `rdf:List`. The `rdfs:range` of `rdf:first` is `rdfs:Resource`.

§ 4.3 `rdf:rest`

`rdf:rest` is an instance of `rdf:Property` that can be used to build descriptions of lists and other list-like structures.

A triple of the form:

```
L rdf:rest O
```

states that there is a rest-of-list relationship between *L* and *O*.

The rdfs:domain of rdf:rest is rdf:List. The rdfs:range of rdf:rest is rdf:List.

§ 4.4 rdf:nil

The resource rdf:nil is an instance of rdf:List that can be used to represent an empty list or other list-like structure.

A triple of the form:

```
L rdf:rest rdf:nil
```

states that *L* is an instance of rdf:List that has one item; that item can be indicated using the rdf:first property.

§ 5. Triple-term based reification

A triple term is a construct defined in [*RDF12-CONCEPTS*], consisting in an RDF triple that is used as a value, specifically, in the object position of another triple. This use represents a reference to a proposition, and, importantly, a triple term does not by itself imply assertion; that is, the proposition it denotes is not necessarily a fact asserted by the RDF graph. This is only the case if that triple is also asserted in the RDF graph — that is, if it is a direct element of that set of triples. Otherwise, it remains unasserted and serves primarily as a reference. This enables the representation of metadata or commentary about triples that may or may not be part of the asserted graph content, including potentially contradictory relationships.

RDF terms that appear in a triple term have the same denotation as when they appear in an asserted triple in the graph. For this reason, triple terms are said to be *transparent*.

A triple term can be the object of a triple whose predicate is rdf:reifies. In such cases, the resulting triple is referred to as a *reifying triple*, and its subject is termed a *reifier*. When the

triple term of a reifying triple also appears in the same graph as an asserted triple, the subset of triples that share the same reifier as subject is called a *triple annotation*.

Triple terms always denote abstract, logical propositions, while reifiers may denote a variety of things related to these propositions (such as a statement or belief that the proposition holds, or an event or circumstance that makes the proposition true). It is therefore expected that triple terms are commonly used as objects of reifying triples, and that reifiers (rather than the triple terms) are used in further descriptions. Because of the diversity of use cases that reifiers aim to serve, the meaning of the `rdf:reifies` property is deliberately generic. Multiple, distinct reifiers may exist for the same proposition (e.g., from different sources or contexts), and, conversely, one reifier may be used for several propositions.

§ 5.1 `rdfs:Proposition`

`rdfs:Proposition` is the class of propositions, simple logical expressions describing a relationship between two entities. `rdfs:Proposition` is an instance of `rdfs:Class` and a subclass of `rdfs:Resource`.

§ 5.2 `rdf:reifies`

`rdf:reifies` is an instance of `rdf:Property` that is used to associate a resource with a proposition.

The `rdfs:domain` of `rdf:reifies` is `rdfs:Resource`. The `rdfs:range` of `rdf:reifies` is `rdfs:Proposition`.

§ 6. Utility Properties

The following utility classes and properties are defined in the RDF core namespaces.

§ 6.1 rdf:value

rdf:value is an instance of rdf:Property that may be used in describing structured values.

rdf:value has no meaning on its own. It is provided as a piece of vocabulary that may be used in idioms such as illustrated in example below:

EXAMPLE 1

```
<http://www.example.com/2002/04/products#item10245>
    <http://www.example.org/terms/weight> [
       rdf:value 2.4 ;
       <http://www.example.org/terms/units> <http://www.example.org
       ] .
```

Despite the lack of formal specification of the meaning of this property, there is value in defining it to encourage the use of a common idiom in examples of this kind.

The rdfs:domain of rdf:value is rdfs:Resource. The rdfs:range of rdf:value is rdfs:Resource.

§ 7. Legacy vocabularies

This section is non-normative.

This section describes vocabularies defined in earlier versions of RDF and RDF Schema. These vocabularies remain in use in certain applications and data sets for historical and compatibility reasons. However, the vocabularies and constructs presented in the preceding sections of this document are generally recommended as preferable alternatives for new developments and modern RDF-based systems.

§ 7.1 Containers

This section is non-normative.

RDF containers are resources that are used to represent collections. The same resource may appear in a container more than once. Unlike containment in the physical world, a container may be contained in itself.

Three different kinds of container are defined. Whilst the formal semantics [*RDF12-SEMANTICS*] of all three classes of container are identical, different classes may be used to indicate informally further information. An `rdf:Bag` is used to indicate that the container is intended to be unordered. An `rdf:Seq` is used to indicate that the order indicated by the numerical order of the container membership properties of the container is intended to be significant. An `rdf:Alt` container is used to indicate that typical processing of the container will be to select one of the members.

Just as a hen house may have the property that it is made of wood, that does not mean that all the hens it contains are made of wood, a property of a container is not necessarily a property of all of its members.

RDF provides vocabularies for describing three classes of containers. Containers have a type, and their members can be enumerated by using a fixed set of *container membership properties*. These properties are indexed by integers to provide a way to distinguish the members from each other, but these indices should not necessarily be thought of as defining an ordering of the container itself; some containers are considered to be unordered.

The RDFS vocabulary adds a generic membership property which holds regardless of position, and classes containing all the containers and all the membership properties.

One should understand this vocabulary as *describing* containers, rather than as a tool for constructing them, as would typically be supplied by a programming language. The actual containers are entities in the semantic universe, and RDF graphs which use the vocabulary simply provide very basic information about these entities, enabling an RDF graph to characterize the container type and give partial information about the members of a container. Since the RDF container vocabulary is so limited, many natural assumptions concerning RDF containers cannot be formally sanctioned by the RDF formal semantics. This should not be

taken as meaning that these assumptions are false, but only that RDF does not formally entail that they must be true.

There are no special semantic conditions on the container vocabulary: the only structure which RDF presumes its containers to have is what can be inferred from the use of this vocabulary and the general RDF semantic conditions. This amounts to knowing the type of a container, and having a partial enumeration of the items in the container. The intended mode of use is that things of type `rdf:Bag` are considered to be unordered but to allow duplicates; things of type `rdf:Seq` are considered to be ordered, and things of type `rdf:Alt` are considered to represent a collection of alternatives, possibly with a preference ordering. If the container is of an ordered type, then the ordering of items in the container is intended to be indicated by the numerical ordering of the container membership properties, which are assumed to be single-valued. However, these informal conditions are not reflected in any formal RDF entailments.

The RDF semantics does not support any entailments which could arise from enumerating the elements of an unordered `rdf:Bag` in a different order. For example,

```
_:xxx rdf:type rdf:Bag .
_:xxx rdf:_1 ex:a .
_:xxx rdf:_2 ex:b .
```

does not entail

```
_:xxx rdf:_1 ex:b .
_:xxx rdf:_2 ex:a .
```

(If this c> valid, then the result of adding it to the original graph would be entailed by the graph, and this would assert that both elements were in both positions. This is a consequence of the fact that RDF is a purely assertional language.)

There is no assumption that a property of a container applies to any of the elements of the container, or vice versa.

There is no formal requirement that the three container classes are disjoint, so that for example it is consistent to assert that something is both an `rdf:Bag` and an `rdf:Seq`. There is no assumption that containers are gap-free, so that for example

```
_:xxx rdf:type rdf:Seq.
_:xxx rdf:_1 ex:a .
```

```
_:xxx rdf:_3 ex:c .
```

does not entail

```
_:xxx rdf:_2 _:yyy .
```

There is no way in RDF to assert that a container contains only a fixed number of members. This is a reflection of the fact that it is always consistent to add a triple to a graph asserting a membership property of any container. And finally, there is no built-in assumption that an RDF container has only finitely many members.

RDF containers are defined by the following classes and properties.

§ 7.1.1 `rdfs:Container`

The `rdfs:Container` class is a super-class of the RDF Container classes, i.e. `rdf:Bag`, `rdf:Seq`, `rdf:Alt`.

§ 7.1.2 `rdf:Bag`

The `rdf:Bag` class is the class of RDF 'Bag' containers. It is a subclass of `rdfs:Container`. Whilst formally it is no different from an `rdf:Seq` or an `rdf:Alt`, the `rdf:Bag` class is used conventionally to indicate to a human reader that the container is intended to be unordered.

§ 7.1.3 `rdf:Seq`

The `rdf:Seq` class is the class of RDF 'Sequence' containers. It is a subclass of `rdfs:Container`. Whilst formally it is no different from an `rdf:Bag` or an `rdf:Alt`, the `rdf:Seq` class is used conventionally to indicate to a human reader that the numerical ordering of the container membership properties of the container is intended to be significant.

§ 7.1.4 rdf:Alt

The rdf:Alt class is the class of RDF 'Alternative' containers. It is a subclass of rdfs:Container. Whilst formally it is no different from an rdf:Seq or an rdf:Bag, the rdf:Alt class is used conventionally to indicate to a human reader that typical processing will be to select one of the members of the container. The first member of the container, i.e. the value of the rdf:_1 property, is the default choice.

§ 7.1.5 rdfs:ContainerMembershipProperty

The rdfs:ContainerMembershipProperty class has as instances the properties rdf:_1, rdf:_2, rdf:_3 ... that are used to state that a resource is a member of a container. rdfs:ContainerMembershipProperty is a subclass of rdf:Property. Each instance of rdfs:ContainerMembershipProperty is an rdfs:subPropertyOf the rdfs:member property.

Given a container *C*, a triple of the form:

```
C rdf:_nnn O
```

where nnn is the decimal representation of an integer greater than 0 with no leading zeros, states that *O* is a member of the container *C*.

Container membership properties may be applied to resources other than containers.

§ 7.1.6 rdfs:member

rdfs:member is an instance of rdf:Property that is a super-property of all the container membership properties i.e. each container membership property has an rdfs:subPropertyOf relationship to the property rdfs:member.

The rdfs:domain of rdfs:member is rdfs:Resource. The rdfs:range of rdfs:member is rdfs:Resource.

§ 7.2 "Old-style" Reification

This section is non-normative.

The intended meaning of this vocabulary is to allow an RDF graph to act as metadata describing other RDF triples.

Consider an example graph containing a single triple:

```
ex:a ex:b ex:c .
```

and suppose that IRI ex:graph1 is used to identify this graph. Exactly how this identification is achieved is external to the RDF model, but it might be by the IRI resolving to a concrete syntax document describing the graph, or by the IRI being the associated name of a named graph in a dataset. Assuming that the IRI can be used to denote the triple, then the reification vocabulary allows us to describe the first graph in another graph:

```
ex:graph1 rdf:type rdf:Statement .
ex:graph1 rdf:subject ex:a .
ex:graph1 rdf:predicate ex:b .
ex:graph1 rdf:object ex:c .
```

The second graph is called a reification of the triple in the first graph.

Reification is not a form of quotation. Rather, the reification describes the relationship between a token of a triple and the resources that the triple denotes. The value of the rdf:subject property is not the subject IRI itself but the thing it denotes, and similarly for rdf:predicate and rdf:object. For example, if the referent of ex:a is Mount Everest, then the subject of the reified triple is also the mountain, not the IRI which denotes it.

Reifications can be written with a blank node as subject, or with an IRI subject which does not identify any concrete realization of a triple, in both of which cases they simply assert the existence of the described triple.

The subject of a reification is intended to denote a concrete realization of an RDF triple, such as a document in a surface syntax, rather than a triple considered as an abstract object. This supports use cases where properties such as dates of composition or provenance information are applied to the reified triple, which are meaningful only when thought of as denoting a particular instance or token of a triple.

A reification of a triple does not entail the triple, and is not entailed by it. The reification only says that the triple token exists and what it is about, not that it is true, so it does not entail the triple. On the other hand, asserting a triple does not automatically imply that any triple tokens exist in the universe being described by the triple. For example, the triple might be part of an ontology describing animals, which could be satisfied by an interpretation in which the universe contained only animals, and in which a reification of it was therefore false.

Since the relation between triples and reifications of triples in any RDF graph or graphs need not be one-to-one, asserting a property about some entity described by a reification need not entail that the same property holds of another such entity, even if it has the same components. For example,

```
_:xxx rdf:type rdf:Statement .
_:xxx rdf:subject ex:subject .
_:xxx rdf:predicate ex:predicate .
_:xxx rdf:object ex:object .
_:yyy rdf:type rdf:Statement .
_:yyy rdf:subject ex:subject .
_:yyy rdf:predicate ex:predicate .
_:yyy rdf:object ex:object .
_:xxx ex:property ex:foo .
```

does not entail

```
_:yyy ex:property ex:foo .
```

§ 7.2.1 `rdf:Statement`

`rdf:Statement` is an instance of `rdfs:Class`. It is intended to represent the class of RDF statements. An RDF statement is the statement made by a token of an RDF triple. The subject

of an RDF statement is the instance of rdfs:Resource identified by the subject of the triple. The predicate of an RDF statement is the instance of rdf:Resource identified by the predicate of the triple. The object of an RDF statement is the instance of rdfs:Resource identified by the object of the triple. rdf:Statement is in the domain of the properties rdf:predicate, rdf:subject and rdf:object. Different individual rdf:Statement instances may have the same values for their rdf:predicate, rdf:subject and rdf:object properties.

RDF statements are not triples in an RDF graph so their values for rdf:predicate do not need to be instances of rdf:Property in that graph, although in most cases they will be.

§ 7.2.2 rdf:subject

rdf:subject is an instance of rdf:Property that is used to state the subject of a statement.

A triple of the form:

```
S rdf:subject R
```

states that *S* is an instance of rdf:Statement and that the subject of *S* is R.

The rdfs:domain of rdf:subject is rdf:Statement. The rdfs:range of rdf:subject is rdfs:Resource.

§ 7.2.3 rdf:predicate

rdf:predicate is an instance of rdf:Property that is used to state the predicate of a statement.

A triple of the form:

```
S rdf:predicate P
```

states that *S* is an instance of rdf:Statement, that *P* is an instance of rdfs:Resource and that the predicate of *S* is *P*.

The rdfs:domain of rdf:predicate is rdf:Statement and the rdfs:range is rdfs:Resource.

7.2.4 rdf:object

rdf:object is an instance of rdf:Property that is used to state the object of a statement.

A triple of the form:

```
S rdf:object O
```

states that *S* is an instance of rdf:Statement and that the object of *S* is *O*.

The rdfs:domain of rdf:object is rdf:Statement. The rdfs:range of rdf:object is rdfs:Resource.

8. RDF Schema summary

This section is non-normative.

The tables in this section provide an overview of the RDF Schema vocabulary.

8.1 RDF classes

Class name	comment
rdfs:Resource	The class resource, everything.
rdfs:Literal	The class of literal values, e.g. textual strings and integers.

rdf:langString	The class of language-tagged string literal values.
rdf:dirLangString	The class of directional language-tagged string literal values.
rdf:HTML	The class of HTML literal values.
rdf:XMLLiteral	The class of XML literal values.
rdf:JSON	The class of JSON literal values.
rdfs:Class	The class of classes.
rdf:Property	The class of RDF properties.
rdfs:Proposition	The class of reified triples.
rdfs:Datatype	The class of RDF datatypes.
rdf:Statement	The class of RDF statements.
rdf:Bag	The class of unordered containers.
rdf:Seq	The class of ordered containers.
rdf:Alt	The class of containers of alternatives.
rdfs:Container	The class of RDF containers.
rdfs:ContainerMembershipProperty	The class of container membership properties, rdf:_1, rdf:_2, ..., all of which are sub-properties of 'member'.
rdf:List	The class of RDF Lists.

§ 8.2 RDF properties

Property name	comment	domain	range
rdf:type	The subject is an instance of a class.	rdfs:Resource	rdfs:Class

rdf:reifies	Associates a resource with a reified triple.	rdfs:Resource	rdfs:Proposition
rdfs:subClassOf	The subject is a subclass of a class.	rdfs:Class	rdfs:Class
rdfs:subPropertyOf	The subject is a subproperty of a property.	rdf:Property	rdf:Property
rdfs:domain	A domain of the subject property.	rdf:Property	rdfs:Class
rdfs:range	A range of the subject property.	rdf:Property	rdfs:Class
rdfs:label	A human-readable name for the subject.	rdfs:Resource	rdfs:Literal
rdfs:comment	A description of the subject resource.	rdfs:Resource	rdfs:Literal
rdfs:member	A member of the subject resource.	rdfs:Resource	rdfs:Resource
rdf:first	The first item in the subject RDF list.	rdf:List	rdfs:Resource
rdf:rest	The rest of the subject RDF list after the first item.	rdf:List	rdf:List
rdfs:seeAlso	Further information about the subject resource.	rdfs:Resource	rdfs:Resource
rdfs:isDefinedBy	The definition of the subject resource.	rdfs:Resource	rdfs:Resource
rdf:value	Idiomatic property used for structured values.	rdfs:Resource	rdfs:Resource
rdf:subject	The subject of the	rdf:Statement	rdfs:Resource

	subject RDF statement.		
rdf:predicate	The predicate of the subject RDF statement.	rdf:Statement	rdfs:Resource
rdf:object	The object of the subject RDF statement.	rdf:Statement	rdfs:Resource

In addition to these classes and properties, RDF uses properties called `rdf:_1`, `rdf:_2`, `rdf:_3`, etc., each of which is both a sub-property of `rdfs:member` and an instance of the class `rdfs:ContainerMembershipProperty`. There is also an instance of `rdf:List` called `rdf:nil` that is an empty `rdf:List`.

§ A. Vocabulary Registry

This section is non-normative.

This section defines a registry for the vocabulary in the RDF core namespaces in accordance with § 6.5 The Registry Track of the W3C Process [*W3C-PROCESS*].

The purpose of this registry is to provide a central index where anyone can find out what vocabulary entries exist in the RDF core namespaces and where they are formally defined, to avoid collisions and duplication.

Each registry table has the following three fields: an IRI, the governing specification, and any relevant notes (such as applicable scope).

The policy for changes to existing entries is as follows:

- Entries can be marked as deprecated but not deleted.
- Entries can be updated to reflect any changes to the governing specification or the notes.

The custodian for all registry tables is the RDF & SPARQL Working Group.

§ A.1 `rdf:` Namespace

This section is non-normative.

The following registry table lists all the IRIs defined in the `rdf:` namespace.

A machine-readable version of this table is available at http://www.w3.org/1999/02/22-rdf-syntax-ns.

IRI	Specification	Notes
rdf:Alt	This document	
rdf:Bag	This document	
rdf:Seq	This document	
rdf:HTML	This document	
rdf:JSON	This document	
rdf:XMLLiteral	This document	
rdf:List	This document	
rdf:Property	This document	
rdf:Statement	This document	
rdf:nil	This document	
rdf:rest	This document	
rdf:dirLangString	This document	

rdf:first	This document	
rdf:langString	This document	
rdf:object	This document	
rdf:predicate	This document	
rdf:subject	This document	
rdf:reifies	This document	
rdf:type	This document	
rdf:value	This document	
rdf:CompoundLiteral	[*JSON-LD11*]	Consideration should be given to the use of rdf:dirLangString. This term is considered archaic but retained for compatibility with JSON-LD.
rdf:direction	[*JSON-LD11*]	Consideration should be given to the use of rdf:dirLangString. This term is considered archaic but retained for compatibility with JSON-LD.
rdf:language	[*JSON-LD11*]	Consideration should be given to the use of rdf:dirLangString. This term is considered archaic but retained for compatibility with JSON-LD.
rdf:PlainLiteral	[*RDF-PLAIN-LITERAL*]	Only used in OWL [*OWL2-OVERVIEW*]

rdf:langRange	[*RDF-PLAIN-LITERAL*]	Only used in OWL [*OWL2-OVERVIEW*]
rdf:PropositionForm	[*RDF-INTEROP*]	Reserved for basic encoding of propositions.
rdf:propositionFormSubject	[*RDF-INTEROP*]	Reserved for basic encoding of propositions.
rdf:propositionFormPredicate	[*RDF-INTEROP*]	Reserved for basic encoding of propositions.
rdf:propositionFormObject	[*RDF-INTEROP*]	Reserved for basic encoding of propositions.

A.2 rdfs: Namespace

This section is non-normative.

The following registry table lists all the IRIs defined in the rdfs: namespace.

A machine-readable version of this table is available at http://www.w3.org/2000/01/rdf-schema.

IRI	Specification
rdfs:Class	This document
rdfs:Container	This document
rdfs:ContainerMembershipProperty	This document
rdfs:Datatype	This document
rdfs:Literal	This document
rdfs:Proposition	This document
rdfs:Resource	This document

`rdfs:comment`	This document
`rdfs:domain`	This document
`rdfs:isDefinedBy`	This document
`rdfs:label`	This document
`rdfs:member`	This document
`rdfs:range`	This document
`rdfs:seeAlso`	This document
`rdfs:subClassOf`	This document
`rdfs:subPropertyOf`	This document

B. Privacy Considerations

This section is non-normative.

See Privacy Considerations in *RDF 1.2 Concepts and Abstract Data Model* [*RDF12-CONCEPTS*].

C. Security Considerations

This section is non-normative.

See Security Considerations in *RDF 1.2 Concepts and Abstract Data Model* [*RDF12-CONCEPTS*].

D. Acknowledgments

This section is non-normative.

§ D.1 Acknowledgments for 2004 Recommendation

This section is non-normative.

The RDF Schema design was originally produced by the RDF Schema Working Group (1997-2000).

David Singer of IBM was the chair of the original RDF Schema group throughout most of the development of this specification; we thank David for his efforts and thank IBM for supporting him and us in this endeavor. Particular thanks are also due to Andrew Layman for his editorial work on early versions of this specification.

The original RDF Schema Working Group membership included:

Nick Arnett (Verity), Dan Brickley (ILRT / University of Bristol), Walter Chang (Adobe), Sailesh Chutani (Oracle), Ron Daniel (DATAFUSION), Charles Frankston (Microsoft), Joe Lapp (webMethods Inc.), Patrick Gannon (CommerceNet), RV Guha (Epinions, previously of Netscape Communications), Tom Hill (Apple Computer), Renato Iannella (DSTC), Sandeep Jain (Oracle), Kevin Jones, (InterMind), Emiko Kezuka (Digital Vision Laboratories), Ora Lassila (Nokia Research Center), Andrew Layman (Microsoft), John McCarthy (Lawrence Berkeley National Laboratory), Michael Mealling (Network Solutions), Norbert Mikula (DataChannel), Eric Miller (OCLC), Frank Olken (Lawrence Berkeley National Laboratory), Sri Raghavan (Digital/Compaq), Lisa Rein (webMethods Inc.), Tsuyoshi Sakata (Digital Vision Laboratories), Leon Shklar (Pencom Web Works), David Singer (IBM), Wei (William) Song (SISU), Neel Sundaresan (IBM), Ralph Swick (W3C), Naohiko Uramoto (IBM), Charles Wicksteed (Reuters Ltd.), Misha Wolf (Reuters Ltd.)

§ D.2 Acknowledgments for RDF 1.1

This section is non-normative.

The current specification is largely an editorial clarification of that design, and has benefited greatly from the hard work of the RDF Core Working Group members, and from

implementation feedback from many members of the RDF Interest Group. In 2013-2014 Guus Schreiber edited this document on behalf of the RDF Working Group to bring it in line with the RDF 1.1 specifications.

§ D.3 Acknowledgments for RDF 1.2

This section is non-normative.

In addition to the editors, the following people have contributed to this specification: Andy Seaborne, Denis Ah-Kang, Gregg Kellogg, Niklas Lindström, Peter F. Patel-Schneider, Pierre-Antoine Champin, and Ted Thibodeau Jr

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

> EDITOR'S NOTE
>
> Recognize members of the Task Force? Not an easy to find list of contributors.

§ E. Change since 2004 Recommendation

This section is non-normative.

Changes for RDF 1.1 Recommendation

- Conversion to ReSpec, including formatting of examples and notes.
- References to RDF 1.0 documents where appropriate replaced by references to RDF 1.1 documents.
- Replaced the term "URI Reference" with the term "IRI".
- Removed discussion about distinction between plain and typed literals, as this distinction is absent in RDF 1.1 and has no technical bearing on RDF Schema.
- Removed the introductory paragraph of Sec. "Reification Vocabulary", as this discussion is not related to the technical content and is irrelevant and confusing now.
- Update of affiliation of the editors.
- Added RDF WG to the Acknowledgments section.
- Renamed the document from "RDF Vocabulary Description Language 1.0: RDF Schema" to "RDF Schema 1.1", as the term Vocabulary Description Language has led to confusion.
- Three paragraphs of the Introduction were left out. These paragraphs described the things that RDF Schema does not do and are now much less relevant than in 2004.
- Added the datatypes `rdf:langString` and `rdf:HTML`.
- Removed Appendix "RDF Schema in RDF/XML". It was informative, but now out of date, in terms of content and in terms of syntax.
- Marked `rdf:HTML` and `rdf:XMLLiteral` as non-normative.
- Removed references to 2004 Primer from Secs. 5.1, 5.2 and 5.4.3. In the latter case the example referred to was moved into this document for readability purposes.

§ F. Changes between RDF 1.1 and RDF 1.2

This section is non-normative.

- Added the datatype `rdf:dirLangString`.
- Added the datatype `rdf:JSON`.
- Added the class `rdfs:Proposition` and the property `rdf:reifies`.
- Reorganization of the document structure.

- Added the vocabulary registry.
- Added the Privacy Considerations and Security Considerations.

§ G. Index

§ G.1 Terms defined by this specification

§ G.2 Terms defined by reference

[*RDF12-CONCEPTS*] defines the following:

directional language-tagged string values

HTML literal values

IRIs

JSON literal values

language-tagged string values

literal

Privacy Considerations

RDF datatype

reifier

reifying triple

Security Considerations

triple annotation

XML literal values

[*RDF12-SEMANTICS*] defines the following:

semantic extension

[*RDF12-TURTLE*] defines the following:

shorthand notation

§ H. References

§ H.1 Informative references

[JSON-LD11]

JSON-LD 1.1. Gregg Kellogg; Pierre-Antoine Champin; Dave Longley. W3C. 16 July 2020. W3C Recommendation. URL: https://www.w3.org/TR/json-ld11/

[OWL2-OVERVIEW]

OWL 2 Web Ontology Language Document Overview (Second Edition). W3C OWL Working Group. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/owl2-overview/

[RDF-INTEROP]

RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. 16 December 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-interop/

[RDF-NOT]

What the Semantic Web can represent. Tim Berners-Lee. W3C. 17 September 1998. W3C-Internal Document. URL: https://www.w3.org/DesignIssues/RDFnot.html

[RDF-PLAIN-LITERAL]

rdf:PlainLiteral: A Datatype for RDF Plain Literals (Second Edition). Jie Bao; Sandro Hawke; Boris Motik; Peter Patel-Schneider; Axel Polleres. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/rdf-plain-literal/

[RDF11-SCHEMA]

RDF Schema 1.1. Dan Brickley; Ramanathan Guha. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf-schema/

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 5 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RDF12-N-QUADS]

RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SEMANTICS]

RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 18 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TRIG]

RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 26 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[W3C-PROCESS]

W3C Process Document. Elika J. Etemad (fantasai); Florian Rivoal. W3C. 18 August 2025. URL: https://www.w3.org/policies/process/

[XMLSCHEMA11-2]

W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. David Peterson; Sandy Gao; Ashok Malhotra; Michael Sperberg-McQueen; Henry Thompson; Paul V.

Biron et al. W3C. 5 April 2012. W3C Recommendation. URL:
https://www.w3.org/TR/xmlschema11-2/

↥